



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

DATES

Date	Section	Time	Demo Due Date	Report Due Date
24-Feb-2026	H21	2:00 to 4:50 PM	06-Mar-2026	10-Mar-2026 11:00 PM
25-Feb-2026	H31		06-Mar-2026	11-Mar-2026 11:00 PM
26-Feb-2026	H41		06-Mar-2026	12-Mar-2026 11:00 PM
27-Feb-2026	H51		06-Mar-2026	13-Mar-2026 11:00 PM

LEARNING OBJECTIVES

This lab provides experience with FreeRTOS queues and UART interfacing using hashing and a simple proof-of-work mechanism. More specifically, the objectives of this lab exercise are:

- To implement UART interfacing for user input and system output via a virtual serial terminal connection between the PC host and the Zybo Z7 board.
- To understand and apply FreeRTOS queues for inter-task communication.
- To gain experience with interfacing software with hardware using both polling and hardware interrupts.
- To gain experience using queues to decouple the execution of data producer and data consumer tasks.
- To learn the basics of hashing functions and their applications.

INTRODUCTION

In this lab you will employ the Zynq-7000 Universal Asynchronous Receiver-Transmitter (UART) Interface in conjunction with the FreeRTOS operating system. This lab exercise is designed to provide a comprehensive understanding of UART communication, especially in the context of real-time operating systems.

The Zynq-7000 SoC (System on Chip) is a powerful platform that combines a high-performance ARM-based processing system (PS) that is integrated on the same chip with programmable logic (PL). This flexible architecture makes it a convenient choice for a wide range of applications, including those requiring real-time processing and versatile communication capabilities. In this lab, we will focus on two primary subjects:

1. Implementing UART Communication using Polling:

In the first part, you will complete an application that utilizes the SHA-256 algorithm to generate a hash string from user-provided username and password information. This application will use a polling method to receive input from the



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

UART device and send data out using a queue. The focus of this section is to learn how to configure, initialize, and interface with the UART peripheral on the Zybo board using a polling loop.

2. Extending Lab 1 (part 3) application with UART communication:

In the second part, you will extend your work from Lab 1, where you previously worked on an application that could send commands to control various peripherals on the board. Building on your knowledge from Part 1, you will integrate UART communication into this system, enabling command input via UART.

3. Developing a UART Driver using Interrupts:

In the final part, you will implement a more advanced UART driver using interrupts. Unlike polling, which requires continuous CPU involvement, interrupt-driven communication allows the system to react asynchronously to incoming data, improving efficiency and responsiveness. This approach is particularly useful for real-time applications where minimizing CPU load is critical.



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

PRE-LAB

Marks 10%

Before coming to your lab session carefully read through the lab manual and review the source code files provided on Canvas. Sufficient documentation is provided within the description of the system's operation and the example code. Reviewing this documentation will not only help you to finish the pre-lab tasks but also enhance your comprehension and readiness for the practical aspects of the lab. To receive full marks the pre-lab work must include the following:

1. Draw a system architecture diagram for Part 1, including the Zybo Z7 board peripherals, the SDK terminal and the FreeRTOS tasks and queues.
2. What are the main differences between a polling-based interface and an interrupt-driven interface?

All pre-lab work is to be completed **individually**. Please submit your documents through the designated submission link on the Canvas lab site. Ensure your submissions are clear, well-organized, and adhere to the guidelines provided to receive full credit.



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

PART 1 - UART COMMUNICATION USING POLLING

Marks 20%

This exercise is designed to introduce students to the UART (Universal Asynchronous Receiver-Transmitter) interface in the Zybo Z7 board using the polling method. Specifically you will implement a text data hashing system, implemented within FreeRTOS. This approach intended to provide a comprehensive learning experience, combining hardware interfacing with software design.

In computer engineering, hashing systems are useful for efficient data management and security. Hashing systems use a one-way function that maps data blocks to equal-length binary hashes. Hashing systems are used in various applications such as:

- Secure password storage, where passwords are stored as hashes rather than as plain text.
- Data deduplication in storage systems, where hashing identifies redundant data.
- Implementing hash tables for quick data retrieval.
- Digital signatures and certificates in cybersecurity.
- Version Control Systems: In version control, hashing is used to identify and track different versions of documents or code. Each version is assigned a unique hash, allowing for the efficient comparison, retrieval, and management of different versions of files. This is particularly important in software development, where tracking changes and maintaining historical versions of code is required.
- Implementing proofs of work (PoW): PoW is a mechanism that secures networks against fraudulent activities and service abuses. In PoW, participants are required to solve computationally intensive puzzles, verified easily through hashing. This process plays a critical role in achieving consensus in decentralized systems, making hashing indispensable in the architecture of modern digital infrastructures.

Polling is a method, sometimes used in UART communication where the CPU repeatedly checks the status of a peripheral device to see if it's ready for data transmission. Unlike interrupt-driven systems, where the device notifies the CPU when it's ready, polling requires the CPU to actively inquire the status of the device.

In our setup, polling is employed to manage data transmission over a UART interface which supports a serial communication protocol. You'll use specific functions like **XUartPs_ReadReg()** for receiving data and **XUartPs_WriteReg()** for sending data. These functions operate in polled mode, meaning they will block the calling task until the data is successfully received or sent. This method is particularly effective in real-time systems, where timely data processing is crucial.



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

System Operation:

- **CLI_Task:** This task reads user input from the serial terminal and directs it to the **Crypto_Task** for further processing waiting for a result message in order to print the results.
- **Crypto_Task:** This task receives user data that needs to be processed and sent back to **CLI_Task** by following these steps:
 - Receives data from **CLI_Task**.
 - Applies a cryptographic hashing function (SHA-256) to the user data.
 - It converts the hash of the into its string representation.
 - If hash verification is required the task will compare the result with the provided hash.
 - Send the result data back to **CLI_Task**.
- **UART_RX_Task:** This task polls the UART registers for capturing and forwarding user inputs which are the keystrokes on the keyboard that enter text into the serial terminal window. The task captures each byte of the user's keyboard input and directs it to **q_rx_byte**.

When you start, create a hardware platform (Use the same hardware configuration as lab 1), then create an application and load the provided resource files, you'll see that the application won't run due to an assertion error, this is because an important queue is missing, your first task is to declare and initialize the queue to eliminate the errors and run the application using the system debugger and connecting to the serial terminal. You should see an initialization message, like in Figure 1 below.

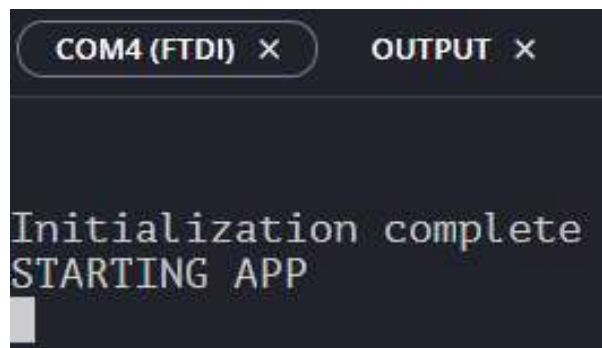


Figure 1: Initial view of the app



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

In this part of the lab, you will finish the design of an application that uses the UART peripheral. Your task is to analyze the behavior of the system, identify the issues, and correct them. You must also explain your reasoning and the changes you make in the report.

When you try to run the application for the first time you will notice that it won't run due to an assertion error, this is because a queue has not been created, your first task is to identify the missing queue and create it with the appropriate size and number of elements. This will eliminate the assertion error and allow the application to start properly. In your report, explain why the assertion occurred and what condition was being checked.

Next, look at **UART_RX_Task** and identify any problems with the polling loop. Fix it and verify that the system responds correctly to received characters. In your report, explain what busy waiting is and why it is problematic in an RTOS-based system.

Modify **UART_TX_Task** so that blocking operations are replaced with non-blocking calls. Explain why blocking operations are not appropriate in a polling-based design.

Next, complete the body of **receive_string()** so that it correctly builds a string from characters received through the UART receive queue. The function must continuously poll the receive queue using a non-blocking operation, read one byte at a time, and append each character to a buffer (buf) until a carriage return ('\r') is detected. When the carriage return is received, the function should terminate the string properly and return.

Your implementation must ensure that the string remains null-terminated at all times and that no buffer overflow can occur. You must check that there is space available before appending a new character, and if the buffer becomes full, additional characters should be ignored until the termination character is received.

Lastly, you must implement your own **print_string()** function and replace all direct calls to **xil_printf()** with this function. The purpose of this change is to ensure that all UART communication flows through your driver layer rather than bypassing it. Your print function should transmit characters one at a time following the same polling-based principles used elsewhere in the system. It must avoid blocking operations and must not introduce busy-wait loops.



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

After completing this step, all UART communication in the application should pass exclusively through the driver abstraction. This ensures consistent behavior, proper use of queues and a clean separation between the application layer and the hardware layer. In your report, explain why routing all communication through the driver layer is good design practice and describe potential issues that could arise if parts of the application bypass the driver and access the hardware directly.

Your report must answer the following questions:

- Why do we use non-blocking operations in polling-based systems?
- Why must a delay be included inside each polling loop?
- How does the polling period affect system responsiveness and CPU usage?
- What problems can arise if a polling loop is implemented incorrectly?
- How can you determine an appropriate polling period for your system?
- What happens if a high-priority task busy-waits in a FreeRTOS system?

Function name	purpose
XUartPs_ReadReg();	Reads a UART register. Returns the values read into an 8-bit variable.
XUartPs_WriteReg();	Writes 8-bit data to a UART register.
XUartPs_IsTransmitFull();	Determine if a byte of data can be sent with the transmitter.
XUartPs_IsReceiveData();	Return TRUE if there is receive data, FALSE otherwise.
xQueueSend();	Sends a message to a queue.
xQueueReceive();	Receives a message from a queue.

Table 1: Useful functions



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

PART 2 - ADDING UART INTERFACE TO LAB 1 PART 3 APPLICATION

Marks 20%

In this part of the lab, you will extend the application developed in Lab 1 Part 3 by integrating UART-based command control. The final system must maintain all original functionality (controlling RGB LED brightness using buttons and displaying keypad input on the SSD) while also accepting commands through the UART interface to control those same peripherals. Specifically, the system must allow commands to change LED brightness, modify LED color, and alter what is displayed on the SSD. The application's behavior from Lab 1 Part 3 must remain fully functional and responsive while adding this new communication layer.

Your first task is to design a command structure that will be used by the UART task to communicate with the RGB and SSD tasks. This structure should clearly represent the type of command being issued and any associated parameters (such as brightness level, color selection, or display value). You are free to define the format and contents of this structure, but it must be flexible enough to support multiple command types in a clean and scalable way.

Next, you must design a method to parse incoming UART strings into your command structure. The parsing approach is entirely up to you; you may use simple string comparison, tokenization, fixed command formats, or any structured format you prefer. However, your solution must be robust, readable, and maintainable. After parsing is complete, the resulting command must be sent to the appropriate task using a FreeRTOS queue.

You must create the necessary queue(s) to transfer parsed commands from the UART task to the RGB and SSD tasks. Each receiving task must implement a polling loop to check for new commands using non-blocking operations. Under no circumstances should your implementation block the system or interfere with the original app behavior.

By the end of this part, your system should behave exactly as it did in Lab 1 Part 3, while additionally responding to UART commands in real time. In your report, explain your command structure design, describe your parsing strategy, justify your queue architecture, and discuss how you ensured that the system remains responsive and free of busy-wait conditions.



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

PART 3 - INTERRUPT DRIVEN INPUT AND OUTPUT

Marks 20%

Interrupts are a critical feature in real-time systems, allowing the processor to respond immediately to certain events. In UART communication, an interrupt is a signal that temporarily halts the currently executing software, allowing a special function, known as an Interrupt Service Routine (ISR), to execute. After the ISR executes, the system resumes executing the interrupted software. This mechanism is more efficient than polling as it allows the CPU to perform other tasks instead of continuously checking the status of the UART. When data is received or ready to be transmitted, the UART generates an interrupt signal, causing the CPU to execute the ISR to handle this data, thus ensuring real-time responsiveness and efficient CPU usage.

The driver's core responsibility includes managing two distinct queues: one for incoming data (the receive direction) and the other for outgoing data (the transmit direction). In the enhanced version of the driver, these queues and implementation details will be abstracted and concealed from the user tasks, emphasizing that certain implementation details are internal to the driver and need not be exposed externally.

In this part, you will complete and enhance an incomplete interrupt-driven driver for the Universal Asynchronous Receiver/Transmitter (UART) within the Zynq-7000 System on Chip (SoC). The primary goal of this exercise is to design an interrupt-driven architecture for effectively receiving and transmitting bytes via the UART interface. Specifically, this involves receiving bytes from the UART and transmitting bytes back to the serial monitor via the same channel.

Three driver functions, named **myReceiveByte()** and **mySendByte()** and **mySendString()**, are to be developed in the file **uart_driver.c**.

This exercise provides practical experience in driver development and data queue management, crucial skills in computer interfacing and embedded systems programming.

In this part of the laboratory, students are required to focus their work on two files: **lab2_part3.c** and **uart_driver.c**.

The **uart_driver.c** file contains the driver software. This software is integral to the exercise as it includes the Interrupt Service Routine (ISR), which is fundamental to the operation of the driver in an interrupt-driven environment. Moreover, this file is where the students will define the two user-defined functions: **mySendByte()** and **myReceiveByte()**. These functions represent the core of the driver software, facilitating improved handling and processing of data through the UART interface.



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

This structure ensures that students are exposed to both the application layer, via **lab2_part3.c**, and the driver layer, through **uart_driver.c**, providing a comprehensive understanding of the software stack and the interactions between different software layers within a real-time operating system environment.

Suggestions on how to successfully complete Part 3 of the lab:

1. Use the declared counter variables to keep track of the number of transmit and receive interrupts.
2. Complete the bodies of the **disableTxEmpty()** and **enableTxEmpty()** functions to control the **TXEMPTY** interrupt.
3. Complete the user-defined function **myReceiveByte()**.
4. Complete the user-defined function **mySendByte()**.
5. Complete the user-defined function **mySendString()**.
6. Test the system by sending strings of varying lengths and observe how many receive and transmit interrupts occur in each case. Evaluate whether these interrupt counts align with your expectations based on how the driver handles each incoming and outgoing byte.
7. Test your system by entering different strings of different lengths and verify the behavior is as expected, comment on similarities and differences in behavior and counts.
8. you can see that we can use a switch to select the value of **sendMethod** in **vBufferReceiveTask**. use the else clause at the end of the task so that when **sendMethod = 1** the task will collect the bytes into an array **txBuffer** until a carriage return “\r” is received, then use **mySendString()** to send the collected data.
9. Next, repeat the same tests from Step 8 and compare the new receive and transmit interrupt counts. Identify any changes in interrupt frequency or patterns, and reflect on why fully queueing the transmission may alter these results.

Function name	purpose
XUartPs_ReadReg();	Reads a UART register. Returns the values read into an 8-bit variable.
XUartPs_WriteReg();	Writes 8-bit data to a UART register.
XUartPs_GetInterruptMask();	Returns the current interrupt mask.
XUartPs_SetInterruptMask();	Sets an Interrupt Mask. The second argument is the mask that contains interrupts to be enabled or disabled. A value of ‘1’ enables the interrupt and ‘0’ disables the interrupt.
xQueueSendToBackFromISR();	Send a message to the back of a queue from the Interrupt Service Routine (ISR).
xQueueReceiveFromISR();	Receives a message from a queue within the Interrupt Service Routine (ISR).

Table 2: Useful functions for completing Part 3



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

QUESTIONS

1. What are the implications for data integrity and system stability in the absence of protection for critical sections within the driver functions?
2. Is there a preferred order for processing interrupts? Briefly analyze how the sequence of handling transmit and receive interrupts in the interrupt service routine (ISR) impacts system performance or reliability. Provide a rationale for why one sequence may be more advantageous over the other in the context of efficient system operation.
3. Explain the importance of disabling transmit interrupts when no data remains to be transmitted. What are the potential effects on the system if transmit interrupts were to remain enabled in such circumstances?
4. Justify the number of interrupts that you found after sending three different sized messages to the Zybo Z7 board and using both transmission methods (**mySendByte** and **mySendString**).
5. Discuss the importance of making queues and other implementation details private and inaccessible to the user in the context of the driver functions. Why is it important to hide implementation details from the users? How does encapsulating these details affect the overall system robustness and reliability?

Additionally, ensure that your main source code, along with the modified `uart_driver.c` file, is submitted as part of your laboratory report. This documentation is important for the comprehensive evaluation of your work.



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

RESOURCES

- [FreeRTOS Documentation](#).
- [Zybo Z7 Reference Manual](#).
- [Secure Hash Standard \(SHS\)](#).



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

REPORT REQUIREMENTS

Cover Page:

- Include the course and lab section numbers, the lab number, and the due date.
- List all student names in your lab team. Please omit ID numbers to respect privacy.

Abstract:

- Summarize the lab's objectives, the hardware used and the exercises in your own words.

Design Section:

- Outline your solutions with a concise explanation of your code for each exercise.
- Include task flow diagrams. Avoid big sections of code snapshots.

Testing Suite:

- Provide a table with test descriptions, expected and actual results and a rationale for each test case.

Conclusion:

- Assess if the given objectives were met and summarize the lab's findings.
- Discuss any issues if the lab was incomplete or unsuccessful. What were the cause(s) of the issues and how could you have resolved them had you had more time.

Formatting and Submission:

- Submit the report as a PDF with lab number and student names in the filename.
- Ensure correct spelling and grammar.
- Use a readable font size (larger than size 10).
- Diagrams may be hand-drawn with a straightedge and scanned at 300 pixel resolution.
- The source code in your report should be syntax-highlighted to enhance readability. Syntax highlighting differentiates the code elements such as keywords comments and strings.
- Adopt a professional format with consistent headings and subheadings.
- Only submit to Canvas your modified source code files in one zip folder. Do not zip the entire workspace.
- Adhere to the submission deadlines as indicated on the first page of the lab handout.



Winter 2026

ECE 315 Computer Interfacing

LAB 2: The Zynq-7000 UART Interface

MARKING SCHEME

Students will be graded based on the form and general quality of the report (clarity, organization, tidiness, spelling, grammar) for the lab. The Pre-lab work is worth 10%. The lab demos are worth 60% and the report is worth 30%. The demonstrations for Parts 1, 2 and 3 are worth 20% each.